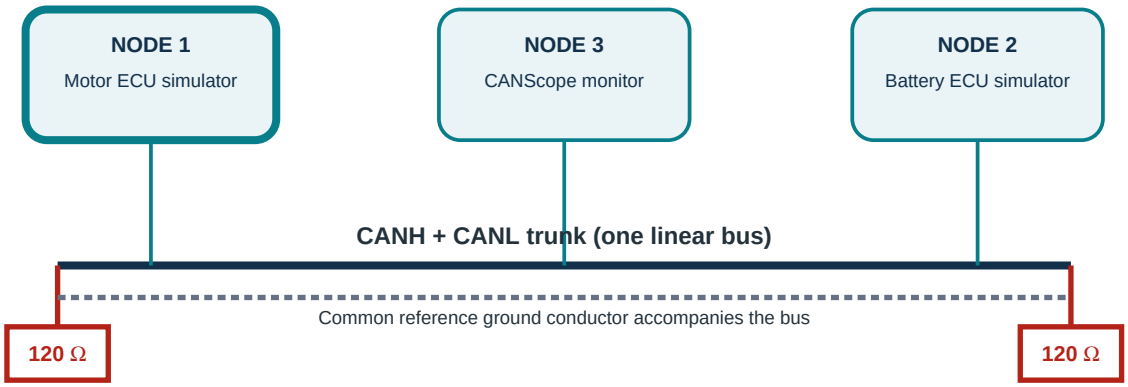
CANScope

Three-node ESP32 + Classical CAN bench build

Physical assembly, wiring planning, inspection, and staged bring-up guide



Exactly two terminators at the two physical ends. Keep stubs short.

READ BEFORE CONNECTING ANY WIRE

This manual is a careful build plan, not a guarantee of zero mistakes. No guide can guarantee zero missed wires without confirming the exact module variants, printed pin labels, ESP32 board revisions, and your physical layout. Photograph and record each item; stop at every marked unresolved assignment. Never energize a STOP-point circuit.

Project roles

Node	Physical role	Build scope
1	Motor ECU simulator	ESP32 + SN65HVD230 CAN interface; role-specific firmware supplied separately
2	Battery ECU simulator	ESP32 + SN65HVD230 CAN interface; role-specific firmware supplied separately
3	CANScope monitor	ESP32 + SN65HVD230, OLED, status LEDs, two buttons, buzzer driver

Revision 1.0 | 16 October 2026 | Based on repository review; no physical hardware inspection or test was performed.

1. What is known, and what must stop

Repository review found candidate CAN assignments but no schematic, board photos, transceiver datasheet, or physical measurements. The source marks all pin choices as assumptions or unverified. Build the mechanical layout and make a written pin plan first; do not wire GPIO signals until the exact map is confirmed.

Connection / value	Repository evidence	Status / action
Node 1 CAN TX/RX	GPIO17 / GPIO16 in legacy sketch/config	WORKING ASSUMPTION only. STOP before GPIO wiring / upload. GPIO16/17 may be occupied by PSRAM on some ESP32-WROVER boards.
Node 2 CAN TX/RX	GPIO17 / GPIO16 in legacy sketch/config	WORKING ASSUMPTION only. STOP before GPIO wiring / upload.
Node 3 CAN TX/RX	GPIO22/21 in brief candidate; 17/16 legacy candidate	UNVERIFIED; firmware intentionally does not start CAN by default. STOP.
Transceiver identity and mode	SN65HVD230 requested by this build brief; repo says part unknown	Match exact module datasheet. STB/RS/EN may change behavior. STOP until labels, supply, and normal-mode wiring are established.
CAN rate/protocol	500 kbit/s legacy assumption; Classical CAN as working assumption	Confirm for test firmware. Do not connect to vehicle/production bus.
Two terminators	Project goal says 2 × 120 Ω	Physical arrangement unverified. Install only at actual two bus ends.
OLED/button/LED/buzzer GPIOs	No pin assignment evidence found	STOP: choose and document after checking board pinout, firmware support, and conflicts.

STOP rule: The tables below intentionally say “TBD - STOP” wherever a GPIO must be selected. Replace a TBD only after reconciling the physical board silk, exact board schematic, and firmware configuration with the software developer. The builder and firmware author must use the same signed-off pin table.

Important board cautions (classic ESP32 assumption only)

Avoid GPIO6-11 (flash). GPIO34-39 are input-only and cannot be CAN TX or output drivers. GPIO0/2/5/12/15 are boot-strapping pins and need board-specific review. GPIO16/17 may be tied to PSRAM on WROVER variants. These cautions do not choose the final pins. Never connect CANH or CANL to ESP32 GPIO.

2. Parts inventory and tools

Qty	Item	Inspection / notes
3	ESP32 DevKit V1, 30-pin	Identify exact module/board revision and both sides. Do not assume every “DevKit V1” silkscreen has identical pin order.
3	SN65HVD230 3.3 V CAN transceiver module	Confirm actual transceiver IC and module schematic. Identify VCC, GND, TXD, RXD, CANH, CANL, STB/RS/EN.
1	SSD1306 0.96-inch I2C OLED	Read labels; verify supply range, address, connector orientation, and if level shifting exists.
1 each	Green, yellow, red LED	Record anode/cathode; each LED requires its own series resistor.
3	220 Ω resistors	For three LEDs, one each. Do not put a single resistor in a shared LED return.
2	Tactile push buttons	Four-leg switch pairs are internally connected; orient across breadboard center gap. Verify with continuity meter.
1	5 V active buzzer	Confirm polarity, current, and whether it is truly active. Never drive directly from GPIO.
1	NPN transistor driver + base resistor	Select a logic-compatible NPN and calculate/check base resistor for buzzer current and GPIO limits. Part values/driver topology need confirmation.
2	120 Ω resistors	CAN end termination only. Measure each resistor before installation.
as needed	Breadboards, jumper wires, USB data cables	Use short CAN stubs; inspect loose strands, damaged insulation, and rail breaks.
1	DMM; optional magnifier	Continuity/ohms before power. Meter must be used in correct jack/mode.
3	USB power/data connections	PC USB for one board at a time during initial programming. No unknown external 5 V injection.

Layout recommendation

Use three labeled breadboard zones, one per node. Mount each ESP32 so its two pin rows straddle the center trench. Put each transceiver near its board, but keep the CAN bus as one linear trunk through the three nodes. Place the two 120 Ω resistors only at the trunk’s two physical ends. Do not make a star. Put Node 3 low-current peripherals on its own breadboard zone.

Prepare a photo record: board top/bottom, transceiver front/back, OLED connector labels, and both sides of each push button. Record markings before assembly.

3. Pin-by-pin wiring plan (sign off before wiring)

This is the connector ledger. It does not assign disputed GPIO numbers. Use one row per physical wire; initial each connection after both ends are checked. “3V3” is the ESP32 regulated 3.3 V pin, only if the board and module documentation permit it.

Node 1 - Motor ECU simulator

ESP32 board pin	To	Wire / verification
3V3	Transceiver VCC (only if confirmed 3.3 V)	TBD; verify module label and datasheet first
GND	Transceiver GND; shared system ground	TBD; continuity check, power off
TBD GPIO - STOP	Transceiver TXD/D	STOP: reconcile chosen CAN TX with firmware
TBD GPIO - STOP	Transceiver RXD/R	STOP: reconcile chosen CAN RX with firmware
Documented mode level - STOP	STB/RS/EN if present	STOP: datasheet determines active level/function
CANH	Bus CANH trunk	Polarity must match all three nodes
CANL	Bus CANL trunk	Polarity must match all three nodes

Node 2 - Battery ECU simulator

ESP32 board pin	To	Wire / verification
3V3	Transceiver VCC (only if confirmed 3.3 V)	TBD; confirm module supply
GND	Transceiver GND; shared system ground	TBD; continuity check, power off
TBD GPIO - STOP	Transceiver TXD/D	STOP: reconcile chosen CAN TX with firmware
TBD GPIO - STOP	Transceiver RXD/R	STOP: reconcile chosen CAN RX with firmware
Documented mode level - STOP	STB/RS/EN if present	STOP: datasheet determines active level/function
CANH	Bus CANH trunk	Same trunk polarity
CANL	Bus CANL trunk	Same trunk polarity

Node 3 - CANScope monitor

ESP32 board pin	To	Wire / verification
3V3	Transceiver VCC and OLED VCC only if each supports 3.3 V	STOP until both variants/supply limits confirmed
GND	Transceiver, OLED, buttons, LED returns, buzzer driver ground	Common reference; avoid ground loops and separate unknown supplies
TBD GPIO - STOP	Transceiver TXD/D	STOP: choose/confirm CAN TX map
TBD GPIO - STOP	Transceiver RXD/R	STOP: choose/confirm CAN RX map
Documented mode level - STOP	STB/RS/EN if present	STOP: use transceiver datasheet/module schematic
TBD GPIO - STOP	OLED SDA	Confirm chosen I2C GPIO and firmware configuration
TBD GPIO - STOP	OLED SCL	Confirm chosen I2C GPIO and firmware configuration

ESP32 board pin	To	Wire / verification
TBD GPIO - STOP	Green LED anode via 220 Ω	LED cathode to GND; confirm pin supports output
TBD GPIO - STOP	Yellow LED anode via 220 Ω	LED cathode to GND; confirm pin supports output
TBD GPIO - STOP	Red LED anode via 220 Ω	LED cathode to GND; confirm pin supports output
TBD GPIO - STOP	Button A one switch side	Other side to GND or documented logic; pull-up strategy is firmware-dependent
TBD GPIO - STOP	Button B one switch side	Other side to GND or documented logic; pull-up strategy is firmware-dependent
TBD GPIO - STOP	NPN base through selected base resistor	Never buzzer directly on GPIO; validate transistor/current design
5V source - STOP until reviewed	Buzzer positive; buzzer negative to transistor collector (typical low-side NPN)	Confirm buzzer polarity/current; emitter to common GND; flyback protection if inductive per datasheet
GND	Transistor emitter and ESP32 GND	Required common reference; if external 5 V used, common ground only after design review
CANH / CANL	Bus trunk	Monitor connection is a branch; termination only if it is physical end

Do not copy a neighboring node GPIO assignment by habit. The pin map is a project-wide interface contract and must match firmware constants on all three boards.

4. Node 3 peripheral circuits

These are circuit patterns, not permission to wire to an unconfirmed GPIO. Before connecting, complete the pin ledger and verify peripheral voltages.

Device	Safe physical pattern	Checks / unresolved items
OLED I2C	ESP32 3V3 -> OLED VCC only if OLED label/datasheet permits; GND -> GND; chosen SDA -> SDA; chosen SCL -> SCL.	TBD SDA/SCL GPIOs, I2C address and firmware. Some modules accept 5 V but pull I2C up to VCC; use 3.3 V logic-safe supply.
Each LED	GPIO -> individual 220 Ω -> LED anode; LED cathode -> GND (or approved sink topology matching firmware).	Confirm LED polarity. One series resistor per LED. Verify current: I approximately (3.3 V - LED Vf)/220 Ω. Keep within GPIO and LED ratings.
Each button	Place across breadboard center gap; one internally connected leg pair to GND, opposite pair to selected GPIO, if firmware uses internal pull-up.	Verify pin pairs by continuity. GPIO and pull-up mode TBD; do not wire both contacts to same side pair.
5 V active buzzer	Use NPN low-side driver: emitter -> GND; collector -> buzzer negative; buzzer positive -> regulated 5 V; ESP32 output -> base resistor -> base.	Confirm transistor pinout and buzzer current. Tie grounds if supplies are separate. Add flyback diode only if buzzer is inductive and per its datasheet; diode cathode toward +5 V. No GPIO direct drive.
Transceiver	ESP32 TX GPIO -> TXD/D; RX GPIO <- RXD/R; module VCC -> confirmed 3.3 V; GND common; CANH/L to bus; mode pin according to exact datasheet.	SN65HVD230 silicon is 3.3 V class, but module wiring/regulators/pullups vary. Read the actual board labels and schematic. Never infer pin order from a web photo.

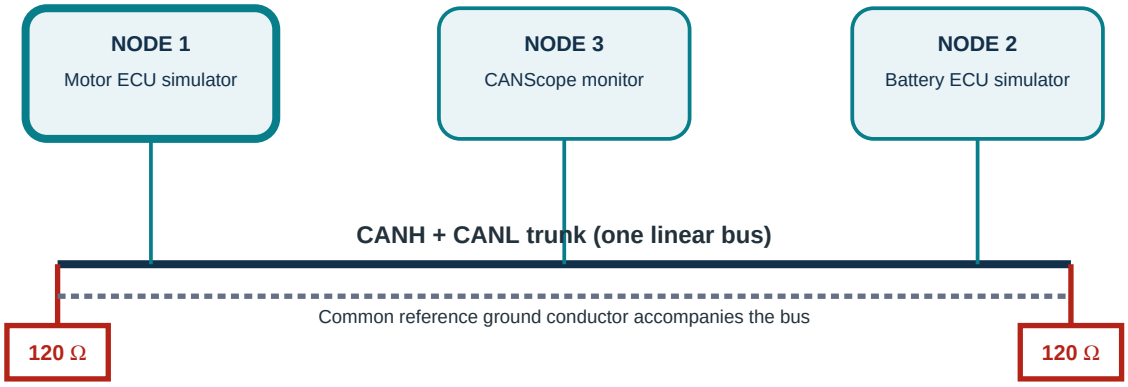
Button orientation sketch: a typical 4-leg tactile switch connects each pair on the same side internally. Insert it over the breadboard center trench so pressing bridges the two sides. Confirm your exact switch with a meter; package variants exist.

Buzzer driver sketch (illustrative low-side NPN): GPIO -- Rbase -- B [NPN] E -- GND; buzzer + to regulated +5 V; buzzer - to C. Grounds must be common. Select the resistor and transistor against the buzzer current and device datasheets.

Power domains

USB usually powers the ESP32 board and may expose USB-derived 5 V and regulated 3.3 V. Do not connect the breadboard 5 V rail back to ESP32 5V/VIN while USB is attached unless the exact board power-path permits it. Never feed 5 V into 3V3. Label rails, check whether breadboard rails are split, and do not bridge split rails by assumption.

5. CAN bus construction and termination



Exactly two terminators at the two physical ends. Keep stubs short.

Wire topology

Route one twisted pair if available as a linear CANH/CANL trunk: physical end terminator -> Node 1 tap -> Node 3 tap -> Node 2 tap -> physical end terminator. The order of the three nodes may change, but the bus remains a line. Keep each tap/stub short, avoid star branches, and keep CANH paired with CANL throughout. A separate ground/reference conductor links node grounds.

Terminators

Fit exactly two 120 Ω resistors, one across CANH-CANL at each physical bus end. Do not put one at every node. If a node is physically at an end, its terminator may be located there. Some transceiver modules include onboard termination or solder jumpers; inspect and measure these before adding external resistors.

Powered-off resistance test

Disconnect USB and all supplies from all nodes. Wait for capacitors to discharge. Measure resistance directly from CANH to CANL on the completed bus. Two 120 Ω terminations in parallel give a nominal reading near 60 Ω. A materially different reading can indicate missing/extra termination, wrong wiring, or a still-connected powered device. Do not “fix” a reading by adding resistors until onboard termination has been ruled out.

Meter reading (unpowered)	Likely interpretation	Action
About 60 Ω	Two 120 Ω end terminations in parallel	Record value; proceed to visual and short checks.
About 120 Ω	One termination present	Find missing physical-end termination; inspect module jumpers.
About 40 Ω	Three 120 Ω terminations in parallel (approx.)	Remove unintended third termination after locating it.
Open / very high	No complete termination path or broken CAN pair	Check bus continuity and end resistors.
Near 0 Ω	Short, wrong measurement points, or severe wiring fault	Do not power. Isolate nodes and locate short.

CANH/CANL are differential bus pins; they are not power pins. Never connect the pair directly to ESP32 pins. Transceiver CANH must connect to trunk CANH and CANL to trunk CANL at every node.

6. Assembly sequence - build unpowered

Step	Action	Check / record
1	Inventory and label nodes N1 Motor ECU, N2 Battery ECU, N3 CANScope. Photograph exact board and module markings.	Complete inventory; capture front/back photos.
2	Mark breadboards and identify split power rails and center trench. Place each ESP32 across trench.	USB connector accessible; no board pins shorted by metal surface.
3	Place each transceiver beside its matching ESP32. Keep signal jumpers short. Do not connect GPIO yet.	Write exact labels in the node pin ledger.
4	Identify CANH/CANL trunk ends and lay the linear bus plus common ground conductor. Leave terminators removable.	Node taps short; no star; label both ends.
5	Measure each 120 Ω resistor individually, then place one at each physical end across CANH-CANL.	Record each measured value and bus end locations.
6	Populate Node 3 OLED, LEDs/resistors, and buttons only after pin map and voltage compatibility are signed off.	One 220 Ω resistor per LED; button across trench.
7	Build the buzzer driver from confirmed transistor pinout and selected resistor; leave 5 V supply disconnected.	Check buzzer polarity/current and ground plan; no direct GPIO path.
8	Connect transceiver power, ground, TXD/RXD, and mode only from approved map/datasheet.	Second-person visual review against signed ledger.
9	Connect CANH, CANL, and ground trunk to all nodes.	Check polarity node-by-node; verify no power rail has been mistaken for CAN.
10	With no power, perform continuity, resistance, and short checks in Section 7.	Record results before any USB insertion.

The repository asks for evidence before values are called verified. Record who checked each wire, date, module identity, continuity or measurement results, and firmware pin map.

7. Pre-power checks and power sequencing

Use a digital multimeter correctly: black lead in COM, red lead in V/Ω jack for continuity/resistance/voltage; never use current mode across a supply. Do resistance checks only with all power removed.

Check	Method (all supplies disconnected)	Pass condition
Board identity	Read board module marking and pin labels; compare manufacturer pinout/schematic.	Exact revision recorded; GPIO ledger signed.
Power shorts	Check 3V3-GND and 5V-GND rails; allow capacitors to settle.	No persistent near-zero short; investigate any low reading.
CAN wiring	Continuity CANH end-to-end; CANL end-to-end; verify no CANH-CANL short except through terminators.	Correct paired conductors; no cross wiring to 5V/3V3.
CAN termination	Measure bus CANH-CANL with all nodes unpowered.	Near 60 Ω expected for two 120 Ω terminations.
Ground commonality	Continuity between all node GND points and intended common reference.	Shared reference; no accidental link to CANH/CANL.
LEDs/buttons	Check LED anode/cathode and resistor in series. Check tactile switch common pairs.	No reversed LED assumptions; button sides identified.
Buzzer	Check transistor pinout vs exact datasheet; inspect for GPIO-to-buzzer direct connection.	Driver is in circuit; GPIO never sources buzzer load directly.
Node mode	Check STB/RS/EN state against exact module schematic/datasheet.	Known normal/active mode for intended test.
USB/power	Remove external 5V during initial USB test unless power path verified.	One known power source per board; avoid backfeeding.

First power-up order

1) Leave CAN trunk disconnected from any vehicle or external network. 2) Power Node 3 alone over USB; keep buzzer 5 V disconnected initially. 3) Check for heat, smell, unexpected current, and measure 3V3/GND and peripheral supply. 4) Power down before any rewiring. 5) Repeat separately for Nodes 1 and 2. 6) Power down all; reconnect the bench bus. 7) Recheck powered-off CANH-CANL resistance. 8) Power the nodes one at a time, observing logs and supply behavior. 9) Add buzzer 5 V only after its driver and common ground have been reviewed.

If a board resets, heats, smells, or a rail collapses: disconnect USB immediately. Do not hot-plug CAN or peripheral wires while powered.

8. Three-node bench bring-up

Stage	Procedure	Expected / stop condition
A. Firmware/pin agreement	Before upload, compare the approved pin ledger with each firmware config. Resolve Node 1/2 GPIO17/16 assumption and Node 3 candidate conflict with software owner. Confirm CAN bitrate and Classical CAN support.	STOP until maps and rate match exactly. Repo Node 3 defaults intentionally leave CAN unconfigured.
B. Single node power	USB-power one board at a time, CAN disconnected. Confirm no shorts or abnormal heating. Check logs for correct node identity and pins.	Power is stable; firmware labels assumptions accurately.
C. Two-node bus	With power off, connect N1 and N2, linear short bus, two end terminators, common ground. Confirm ~60 Ω unpowered. Power and run agreed test firmware only.	Known test frame exchanged; no uncontrolled transmit or vehicle connection.
D. Add monitor	Power down; insert N3 as a short stub or end node if physically at end. Preserve exactly two end terminators. Recheck ~60 Ω. Power monitor in documented listen-only mode if supported.	N3 sees raw traffic; no unapproved frame transmission.
E. Add peripherals	Power down; attach OLED/LED/buttons per signed ledger. Verify display and each indicator independently. Add buzzer supply last.	No peripheral affects CAN or resets board.
F. Evidence	Record bus resistance, supply voltages, firmware versions, GPIOs, bitrate, frame IDs, errors, and photos.	Only measured/observed items move from assumption to verified.

CAN acknowledgement note

A CAN transmitter may need another active node to acknowledge a frame. A monitor configured listen-only does not acknowledge. Bring-up success depends on the actual firmware mode and test setup; do not diagnose wiring solely from an ACK error. Keep the monitor passive for observation unless the test plan explicitly requires normal mode.

This is an isolated bench simulator, not a vehicle connection guide. Do not connect to a real vehicle/industrial bus without a separately reviewed electrical, protocol, and safety plan.

9. Troubleshooting and stop conditions

Symptom	Likely checks	Safe response
No CAN frames	Wrong TX/RX mapping, transceiver standby, bitrate mismatch, CANH/L reversed, missing common ground, no active ACK partner.	Power off; validate labels/map/mode and wiring. Do not swap wires live.
Bus-off/errors	Bitrate mismatch, bad termination, long stubs, poor wiring, wrong mode, short/open pair.	Disconnect power; verify ~60 Ω and linear topology, then isolate nodes.
Resistance not near 60 Ω	Missing/excess termination, onboard module termination, powered device, measurement error.	Remove all power and inspect each termination source before changing.
OLED blank	Wrong SDA/SCL GPIO, supply, polarity, I2C address, connector pin order.	Power down; verify labels and pin map. Never try 5 V on SDA/SCL as a diagnostic.
ESP32 boot failure	Strapping pin held wrong, peripheral loading boot pin, USB power issue.	Power off; inspect actual board pin use and remove peripheral loads for diagnosis.
LED stays off/on	LED reversed, wrong GPIO, resistor not in series, firmware polarity.	Power off; check anode/cathode and chosen pin.
Button always pressed	Button legs placed on same internal pair, wrong pull-up/pull-down, pin conflict.	Power off; identify contact pairs with continuity meter and check firmware input mode.
Buzzer silent / board resets	Transistor pin order wrong, insufficient drive, supply sag, no common ground, wrong active buzzer.	Disconnect buzzer supply; verify current rating and driver design. Never bypass transistor to test.
Hot component, odor, smoke	Short, reversed supply, overcurrent, wrong module voltage.	Disconnect all power immediately; do not touch hot components; inspect before reuse.

Do not “try random GPIOs” or move CANH/CANL while energized. Every change should be logged, performed unpowered, and followed by continuity/resistance checks.

10. Final wire-by-wire sign-off checklist

Fill this in before first power. Add a photo showing each completed node and the entire CAN trunk. Any blank STOP field means wiring/firmware bring-up is not authorized by this checklist.

Sign-off item	Recorded value / initials
ESP32 exact module + board revision, N1/N2/N3	_____
Transceiver module identification + pin labels, each node	_____
Signed CAN TX/RX GPIOs and firmware config match	N1: _____ N2: _____ N3: _____
Node 3 I2C SDA/SCL GPIOs + OLED supply/address	SDA: _____ SCL: _____ Supply: _____ Address: _____
LED GPIOs and resistor/polarity check	Green: _____ Yellow: _____ Red: _____
Button GPIOs and verified switch contact pairs	A: _____ B: _____
Buzzer type/current, transistor part/pinout, base resistor, 5 V source	_____
Transceiver normal-mode STB/RS/EN documented	N1: _____ N2: _____ N3: _____
Two 120 Ω terminators located at physical bus ends	End A location: _____ End B location: _____
Unpowered CANH-CANL bus resistance	_____ Ω; date/time: _____
Ground continuity + CANH/CANL polarity checks	Checked by: _____
No CAN-to-GPIO, no 5V-to-3V3, no GPIO direct buzzer drive	Checked by: _____
CAN bitrate/protocol and firmware node identities aligned	_____
Photos/notes saved; test bus isolated from vehicle	Location: _____

Repository source notes

Reviewed project files: docs/hardware-status.md; docs/wiring/node-3-pin-map.md; firmware/node1/include/node1_hw_config.h; firmware/node2/include/node2_hw_config.h; firmware/node3/include/node3_hw_config.h; README and project brief. Repository records Node 1/2 GPIO17/16 and 500 kbit/s as working assumptions; Node 3 GPIO22/21 versus GPIO17/16 as unresolved candidates; transceiver/module, mode pin, termination, and physical wiring as unverified. No hardware tests were performed for this manual.

Completion status: This manual provides a safe planning and inspection sequence. It does not certify a final GPIO pinout. All TBD/STOP entries must be resolved with exact hardware evidence and firmware agreement before wiring those pins or uploading application firmware.